

F32a Instruction Set Architecture

Справочная документация

32-разрядная стековая архитектура, вдохновлённая F18a

greenarraychips.com - F18a reference

Содержание

1	Обзор архитектуры	2
1.1	Ключевые характеристики	2
2	Регистры и состояние	2
2.1	Регистры	2
2.2	Стек данных	2
2.3	Стек возвратов	2
3	Флаги	2
3.1	Флаг переноса (C)	2
3.2	Extended Arithmetic Mode (EAM)	3
4	Инструкции	3
4.1	Инструкции перемещения данных	4
4.2	Арифметические инструкции	5
4.3	Побитовые инструкции	6
4.4	Инструкции управления стеком	6
4.5	Инструкции управления потоком	6
4.6	Инструкции обмена между стеками	7
5	Особенности архитектуры	8
5.1	Поведение флага переноса C	8
5.2	Автоинкремент в fetch/store	8
5.3	Двойная роль стека возвратов	8
5.4	Стековая операция over	8
5.5	Алгоритм умножения ++	9
5.6	Алгоритм деления +/	9
5.7	Memory-mapped I/O	9
6	Краткая справочная таблица	9

1 Обзор архитектуры

F32a — 32-разрядная стековая архитектура образовательного назначения, вдохновлённая процессором GreenArrays F18a. Программирование ведётся в стиле Forth: основным хранилищем операндов служит стек данных, а не набор регистров.

1.1 Ключевые характеристики

- Два первичных стека: **стек данных** и **стек возвратов**.
- Два регистра общего назначения: **A** и **B**.
- Стеко-ориентированные операции упрощают многие задачи.
- **Extended Arithmetic Mode (EAM)** — режим сложения с переносом.
- **Флаг переноса (C)** для арифметических операций.
- Memory-mapped I/O.
- Размер инструкции: 1 байт — опкод; по 4 байта — каждый аргумент.
- Комментарии в ассемблерном коде начинаются с символа `\`.

2 Регистры и состояние

2.1 Регистры

Регистр	Отображение	Описание
A	A:dec, A:hex	Регистр общего назначения. Используется как адресный указатель в инструкциях @, @+, !, !+, а также может хранить произвольные данные (a! / a).
B	B:dec, B:hex	Вторичный адресный регистр. Используется в @b и !b, а также как указатель делителя в операции +/- (mem[B] — делитель).

Таблица 1: Регистры общего назначения

2.2 Стек данных

2.3 Стек возвратов

3 Флаги

3.1 Флаг переноса (C)

Отображение: C (0 — сброшен, 1 — установлен)

- Устанавливается инструкцией + при переполнении 32-разрядного результата.
- Сохраняется только инструкцией dup (единственное исключение).

Слот	Отображение	Описание
T	T:dec, T:hex	Top of stack — вершина стека данных. Используется в качестве первого операнда для всех арифметических и логических инструкций.
S	S:dec, S:hex	Second — второй элемент стека данных. Используется как второй операнд.
весь стек	stack:dec, stack:hex	Полный снимок стека данных для отладки.

Таблица 2: Состояние стека данных

Слот	Отображение	Описание
R	R:dec, R:hex	Top of return stack — вершина стека возвратов. Хранит адрес возврата при вызове подпрограммы или счётчик итераций для <code>next</code> .
весь стек	rstack:dec, rstack:hex	Полный снимок стека возвратов для отладки.

Таблица 3: Состояние стека возвратов

- **Сбрасывается** всеми инструкциями, помещающими значение в стек данных, за исключением `dup`.

3.2 Extended Arithmetic Mode (EAM)

Отображение: EAM (0 — выключен, 1 — включён)

- Управляется инструкцией `eam`: значение снимается с вершины стека данных и присваивается флагу.
- При `EAM = 1` инструкция + выполняет сложение с учётом текущего флага переноса `C`:

$$T \leftarrow \text{pop}() + \text{pop}() + C$$

- При `EAM = 0` флаг `C` в сложении не участвует.
- Используется для реализации многоразрядного сложения (64-битного и шире) путём цепочки инструкций + с включённым EAM.

4 Инструкции

В таблицах ниже используются следующие обозначения:

- `pop()` — снять значение с вершины стека данных.
- `push(x)` — положить `x` на вершину стека данных.
- `top()` — прочесть вершину без снятия.

- `mem[addr]` — обращение к памяти по адресу.
- `rPop()` / `rPush(x)` — операции со стеком возвратов.
- `P` — программный счётчик.
- `[n]` — бит с номером `n`.

4.1 Инструкции перемещения данных

Синтаксис	Название	Операция
<code><value></code> <code>lit <value></code>	Literal	<code>push(<value>)</code> Помещает непосредственное значение на вершину стека. Ключевое слово <code>lit</code> необязательно; допустимы десятичные числа, hex (<code>0xFF</code>) и символьные литералы (<code>'A'</code>).
<code>@p <addr></code>	Fetch from address	<code>push(mem[<addr>])</code> Читает значение из памяти по явному адресу.
<code>@</code>	Fetch	<code>push(mem[A])</code> Читает значение из памяти по адресу в регистре <code>A</code> .
<code>@+</code>	Fetch plus	<code>push(mem[A]); A ← A + 1</code> Читает по адресу <code>A</code> , затем инкрементирует <code>A</code> (автоинкремент).
<code>@b</code>	Fetch B	<code>push(mem[B])</code> Читает значение из памяти по адресу в регистре <code>B</code> .
<code>!p <addr></code>	Store to address	<code>mem[<addr>] ← pop()</code> Записывает вершину стека в память по явному адресу.
<code>!</code>	Store	<code>mem[A] ← pop()</code> Записывает вершину стека в память по адресу в <code>A</code> .
<code>!+</code>	Store plus	<code>mem[A] ← pop(); A ← A + 1</code> Записывает в память по адресу <code>A</code> , затем инкрементирует <code>A</code> .
<code>!b</code>	Store B	<code>mem[B] ← pop()</code> Записывает вершину стека в память по адресу в <code>B</code> .
<code>a!</code>	A-Store	<code>A ← pop()</code> Загружает значение с вершины стека в регистр <code>A</code> .

Синтаксис	Название	Операция
b!	B-Store	$B \leftarrow \text{pop}()$ Загружает значение с вершины стека в регистр B.
a	A-Fetch	$\text{push}(A)$ Помещает текущее значение регистра A на стек данных.

4.2 Арифметические инструкции

Синтаксис	Название	Операция
+	Add	Без ЕАМ: $\text{push}(\text{pop}() + \text{pop}())$, обновить C С ЕАМ: $\text{push}(\text{pop}() + \text{pop}() + C)$, обновить C Единственная инструкция, которая <i>устанавливает</i> флаг C.
++	Multiply step	Один шаг умножения (алгоритм сдвига и сложения): $T \leftarrow T + (\text{if } A[0] \text{ then } S \text{ else } 0);$ $A \leftarrow A \gg 1;$ $\text{if } T[0] \text{ then } A[31] \leftarrow 1 \text{ else } A[31] \leftarrow 0;$ $T \leftarrow T \gg 1.$ Повторяется 32 раза для полного 32×32-бит умножения.
+/	Divide step	Один шаг восстанавливающего деления: Делимое — A, делитель — $\text{mem}[B]$, частичный остаток — S, частичное частное — T. $S \leftarrow S \ll 1;$ $\text{if } A[31] \text{ then } S[0] \leftarrow 1;$ $A \leftarrow A \ll 1;$ $T \leftarrow T \ll 1;$ $\text{if } S \geq \text{mem}[B] \text{ then } \{ S \leftarrow S - \text{mem}[B];$ $T[0] \leftarrow 1 \}.$
2*	Left shift	$\text{push}(\text{pop}() \ll 1)$ Сдвиг влево на 1 бит (умножение на 2).
2/	Right shift	$\text{push}(\text{pop}() \gg 1)$ Арифметический сдвиг вправо на 1 бит с сохранением знака (деление на 2).

Синтаксис	Название	Операция
inv	Invert	<code>push(~pop())</code> Побитовое инвертирование всех 32 бит вершины стека.
eam	Set EAM	<code>EAM ← pop()</code> Снимает значение с вершины стека и устанавливает флаг EAM.

4.3 Побитовые инструкции

Синтаксис	Название	Операция
and	And	<code>push(pop() & pop())</code> Побитовое И двух вершин стека.
xor	Xor	<code>push(pop() ^ pop())</code> Побитовое исключающее ИЛИ двух вершин стека.

4.4 Инструкции управления стеком

Синтаксис	Название	Операция
drop	Drop	<code>pop()</code> Удаляет значение с вершины стека данных (без использования).
dup	Dup	<code>push(top())</code> Дублирует вершину стека. Особенность: единственная инструкция, помещающая значение на стек, которая <i>не сбрасывает</i> флаг C.
over	Over	<code>t ← pop(); s ← pop(); push(s); push(t); push(s)</code> Копирует второй элемент стека на вершину, сохраняя оба исходных значения.

4.5 Инструкции управления потоком

Синтаксис	Название	Операция
;	Return	$P \leftarrow rPop()$ Возврат из подпрограммы: адрес возврата снимается со стека возвратов.
<label>	Call	$rPush(P); P \leftarrow \text{<label>}$ Вызов подпрограммы: текущий счётчик команд сохраняется в стек возвратов, управление передаётся на метку.
<label> ;	Jump	$P \leftarrow \text{<label>}$ Безусловный переход. Используется форма с суффиксом ; для хвостового вызова (tail call).
next <label>	Next	if $R \neq 0$: $R \leftarrow R - 1; P \leftarrow \text{<label>}$ else: $rPop()$ Декремент R и переход на метку, если $R \neq 0$; иначе снимает R со стека возвратов и продолжает выполнение.
if <label>	If	if $pop() == 0$: $P \leftarrow \text{<label>}$ Переход, если вершина стека равна нулю (снимает значение с T).
-if <label>	Minus If	if $pop() >= 0$: $P \leftarrow \text{<label>}$ Переход, если вершина стека неотрицательна (снимает значение с T).
halt	Halt	Останавливает выполнение машины.

4.6 Инструкции обмена между стеками

Синтаксис	Название	Операция
r>	RIntoT	$push(rPop())$ Перемещает вершину стека возвратов на стек данных.
>r	TIntoR	$rPush(pop())$ Перемещает вершину стека данных на стек возвратов.

5 Особенности архитектуры

5.1 Поведение флага переноса C

Флаг C — единственный бит состояния, влияющий на результат арифметической операции (при включённом EAM). Его поведение строго определено:

- **Сброс:** любая инструкция, помещающая значение на стек данных, автоматически сбрасывает C — за *единственным* исключением.
- **Сохранение:** dup не изменяет C.
- **Установка:** + устанавливает C, если результат сложения не уместится в 32 бита.

Таким образом, последовательность для 64-битного сложения выглядит так:

\	:	[lo_a, hi_a]	[lo_b, hi_b]	
1 eam	\		EAM	
lo_a lo_b +	\		32	, C <-
hi_a hi_b +	\		32	+ C

5.2 Автоинкремент в fetch/store

Инструкции @+ и !+ инкрементируют регистр A *после* операции с памятью. Это позволяет эффективно обходить массивы в памяти без явных инструкций сложения:

base_addr a!	\	A <-
@+	\	T <- mem[A], A <- A+1
@+	\	T <- mem[A], A <- A+1
@+	\	T <- mem[A], A <- A+1

5.3 Двойная роль стека возвратов

Стек возвратов выполняет две функции:

1. Хранит адреса возврата при вызовах подпрограмм (call / ;).
2. Служит счётчиком цикла для инструкции next: значение R декрементируется при каждой итерации и снимается со стека при достижении нуля.

Инструкции >r и r> позволяют явно перемещать данные между стеками, однако их нужно применять с осторожностью внутри вызовов подпрограмм, чтобы не испортить адреса возврата.

5.4 Стековая операция over

Семантика over описывается последовательностью:

$$\dots, s, t \xrightarrow{\text{over}} \dots, s, t, s$$

Формально: снять T, снять S, поместить S, поместить T, поместить S. Результат: исходный S оказывается и под T, и на вершине.

5.5 Алгоритм умножения $++$

Инструкция $++$ реализует один шаг алгоритма умножения *сдвиг-и-сложение*. Полное 32×32 -бит умножение выполняется циклом из 32 итераций:

```

\          S      A;          -> T (
          32      )
0          | T <- 0 (          )
32 >r      |
loop: ++
next loop

```

На каждой итерации младший бит A ($A[0]$) управляет условным прибавлением S к T; оба операнда сдвигаются вправо.

5.6 Алгоритм деления $+/$

Инструкция $+/$ реализует один шаг *восстанавливающего деления*. Делитель хранится в `mem[B]`, что позволяет не занимать стек данных под константу. Полное деление требует 32 итераций аналогично умножению. Подробный пример на Python: <https://github.com/ryukzak/wrench/blob/master/example/step-by-step-div.py>

5.7 Memory-mapped I/O

Внешние устройства доступны через обычные инструкции чтения и записи по фиксированным адресам. Нет отдельного пространства I/O или специальных инструкций — любой адрес в карте памяти может соответствовать периферии.

6 Краткая справочная таблица

Мнемоника	Краткая операция	Примечание
<code>lit v</code>	<code>push(v)</code>	Сбрасывает C
<code>@p a</code>	<code>push(mem[a])</code>	Сбрасывает C
<code>@</code>	<code>push(mem[A])</code>	Сбрасывает C
<code>@+</code>	<code>push(mem[A]); A++</code>	Сбрасывает C
<code>@b</code>	<code>push(mem[B])</code>	Сбрасывает C
<code>!p a</code>	<code>mem[a] <- pop()</code>	—
<code>!</code>	<code>mem[A] <- pop()</code>	—
<code>!+</code>	<code>mem[A] <- pop(); A++</code>	—
<code>!b</code>	<code>mem[B] <- pop()</code>	—
<code>a!</code>	<code>A <- pop()</code>	—
<code>b!</code>	<code>B <- pop()</code>	—
<code>a</code>	<code>push(A)</code>	Сбрасывает C
<code>+</code>	<code>push(pop()+pop()+C_{ЕАМ})</code>	Устанавливает C
<code>++</code>	шаг умножения	см. §4.5
<code>+/</code>	шаг деления	см. §4.6
<code>2*</code>	<code>push(pop() « 1)</code>	Сбрасывает C

Мнемоника	Краткая операция	Примечание
2/	push(pop() » 1) (арифм.)	Сбрасывает C
inv	push(~pop())	Сбрасывает C
eam	EAM <- pop()	—
and	push(pop() & pop())	Сбрасывает C
xor	push(pop() ^ pop())	Сбрасывает C
drop	pop()	—
dup	push(top())	C не изменяется
over	push(second())	Сбрасывает C
;	P <- rPop()	Return
<lbl>	rPush(P); P <- lbl	Call
<lbl> ;	P <- lbl	Jump (tail call)
next l	if R!=0: R-; P<-1	Loop
if l	if pop()==0: P<-1	Branch on zero
-if l	if pop()>=0: P<-1	Branch on nonneg
halt	остановка	—
r>	push(rPop())	Сбрасывает C
>r	rPush(pop())	—